Neurocomputing 672 (2026) 132779

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

AEMC: Aggregator-efficient model compression for streamlining LLMs

Mingrui Zhou [a], Rui Mao [b], Somayajulu Sripada [a], Xiao Li [a,*]

[a] Department of Computing Science, University of Aberdeen, Aberdeen AB24 3FX, United Kingdom
[b] College of Computing and Data Science, Nanyang Technological University, 639798, Singapore

ARTICLE INFO

Communicated by N. Passalis

Keywords:
Deep pruning
Model compressions
Natural language processing (NLP)
Machine learning

ABSTRACT

Large language models (LLMs) are increasingly used in practical applications via simple prompt-based interfaces, but their high inference cost limits deployment. Structured pruning offers a promising solution, yet most effective methods require post-training, which poses barriers in real-world scenarios. We identify amplitude mismatch, a hidden state norm discrepancy caused by residual connections, as a key reason for pruning-induced degradation. To address this, we propose Aggregator-Efficient Model Compression (AEMC), a training-free pruning framework that inserts lightweight Aggregator layers to restore residual amplitude and preserve information flow. Without any fine-tuning, AEMC consistently outperforms all existing training-free pruning methods and achieves state-of-the-art performance even compared to post-trained baselines. AEMC offers a practical path toward efficient and scalable LLM deployment in resource-constrained settings.

1. Introduction

Large language models (LLMs) based on the Transformer architecture [1], such as GPT-style models [2] and DeepSeek [3], have demonstrated impressive performance on a wide range of general-purpose tasks, including complex reasoning, summarisation and question answering [4]. In practice, however, a substantial portion of real-world applications demand much simpler but highly repetitive capabilities, for instance sentiment analysis in user comments, detection of abnormal or toxic content, or filtering spam and advertisements on online platforms. These applications are often small- to medium-scale (SMA) services, with tight latency and cost constraints and domain-specific requirements.

Traditionally, such tasks were addressed by training separate lightweight models tailored to each application. This approach creates a significant barrier for teams without strong machine learning expertise or access to specialised hardware. In contrast, LLMs combined with prompt engineering have enabled a low-friction development paradigm: non-expert developers can rapidly build domain-specific solutions by designing prompts instead of training task-specific models. This usability has driven the rapid adoption of LLMs across many industries.

The downside is that LLM inference is orders of magnitude more expensive than that of conventional NLP models [3]. Running full-scale LLMs for production services can quickly become prohibitively costly, especially when models must be replicated across multiple services or customer deployments [36]. Consequently, there is a growing demand for more compact and efficient alternatives, which has motivated research into model compression techniques such as distillation [5–7], quantisation [8–10], and pruning [11–15].

Among these, structured pruning is particularly attractive for deployment. By removing entire layers or residual blocks, deep pruning reduces the depth of the computation graph, directly saving floating-point operations, decoding latency and KV-cache memory in a hardware-agnostic manner. Importantly, the resulting model remains a standard FP16/BF16 network that can be deployed with existing inference stacks. In principle, one might expect that pruning should behave “gracefully”: removing more layers should gradually reduce accuracy, but the model should remain functional as long as some capacity is preserved.

In practice, this is not what we observe. Empirically, structured pruning often exhibits a sharp collapse: once the pruning ratio exceeds a certain threshold, model performance drops abruptly rather than degrading smoothly [15]. This behaviour is particularly striking given the architecture of LLMs. Because the residual pathway forms a fast information highway that spans all layers, each residual block is intuitively expected to refine the accumulated representation from earlier layers. From this perspective, removing a subset of blocks should primarily reduce prediction precision, not completely destroy the model’s ability to follow instructions or produce coherent outputs.

This mismatch between architectural intuition and empirical behaviour raises a fundamental question: **Why does deep structured pruning of LLMs so often lead to model collapse, rather than a**

* Corresponding author.
Email address: xiao.li@abdn.ac.uk (X. Li).

https://doi.org/10.1016/j.neucom.2026.132779
Received 31 October 2025; Received in revised form 11 January 2026; Accepted 17 January 2026
Available online 20 January 2026
0925-2312/© 2026 Published by Elsevier B.V.

smooth trade-off between depth and accuracy? Existing pruning methods largely sidestep this question by relying on some form of post-training—for example, inserting new lightweight modules that are fine-tuned after pruning, or merging layers with calibration-based optimisation [11,13]. While effective, these approaches reintroduce many of the practical difficulties that LLMs were supposed to avoid: they require additional data, hyperparameter tuning and GPU resources, and they complicate the deployment pipeline. For SMA scenarios in particular, such methods may be impractical.

In this work, we investigate a training-free alternative and argue that the key to understanding pruning-induced collapse lies in the norm dynamics of the residual stream. We conduct a systematic analysis of hidden-state behaviour in several LLaMA and Baichuan models and make two consistent observations: (i) the direction of the residual stream changes only mildly across layers (high input–output cosine similarity), whereas (ii) its norm grows steadily with depth. These findings suggest that many residual blocks act primarily as amplitude modulators, scaling the magnitude of the shortcut vector while only slightly adjusting its direction.

Building on this view, we introduce the notion of amplitude mismatch: when layers are removed, the ratio between the norms of the shortcut and residual branches is distorted, so that new residual contributions can overwrite, rather than refine, the information accumulated by previous layers. We show that this mismatch is the core mechanism behind the catastrophic degradation observed in deep structured pruning. When we correct amplitude mismatch, pruning again behaves as expected: removing more layers leads to a gradual loss of accuracy, but the model no longer collapses.

To address amplitude mismatch without any retraining, we propose Aggregator-Efficient Model Compression (AEMC), a training-free pruning framework that inserts simple Aggregator (AGG) layers at pruned positions. Each AGG layer is a parameter-free scalar transformation of the form

$$h_i \mapsto \lambda_i h_i$$

applied to the shortcut vector at layer i. Here, λ_i is a layer- and task-specific scalar estimated from a handful of calibration examples; it is chosen so that the shortcut norm after pruning matches the norm that the dense model would have produced. In other words, AGG layers preserve the original shortcut–residual mixing ratio while keeping the direction of the representation intact.

AEMC is designed to be both methodologically grounded and practically simple: (1) It directly targets the residual norm distortion identified as the source of pruning collapse; (2) It does not introduce any learnable parameters or require gradient-based optimisation; (3) It uses only a few examples per task for configuration, and the same calibration process also reveals which layers are unsafe to prune.

Extensive experiments on multiple open-source LLMs (LLaMA 2/3.1 and Baichuan 2) and diverse NLP benchmarks show that AEMC consistently outperforms existing training-free pruning methods and, remarkably, matches or surpasses several post-training baselines, while requiring no fine-tuning and no additional training data.

Our contributions can be summarised as follows:

- **Mechanism**. We provide a theoretical and empirical analysis of amplitude mismatch in Transformer residual connections and show that it is the core mechanism behind catastrophic degradation in deep structured pruning: deleting blocks can drastically alter the norm ratio between shortcut and residual branches, leading to uncontrolled overwriting of accumulated information.
- **Method**. We propose AEMC, a strictly training-free framework that replaces each removed residual block with an optimal scalar Aggregator $h_i \mapsto \lambda_i h_i$. Each λ_i is estimated per layer and per task from a small calibration set and is used both to reconstruct the original residual amplitude and to identify layers that should not be pruned.
- **Effectiveness and practicality**. Across several LLaMA and Baichuan models, AEMC achieves state-of-the-art performance among training-free pruning methods and competes with post-training approaches, while being extremely simple to deploy in industrial settings: no fine-tuning, no new parameters, and minimal task-specific calibration.

2. Background: Residual dynamics in LLMs

We begin by formalising the residual structure of LLMs and summarising key empirical properties of the residual stream. These observations motivate our definition of amplitude mismatch in Section 3.

2.1. Residual formulation

Consider an LLM with an embedding layer, followed by L Transformer layers and a final output head. Let h_0 denote the token-wise hidden states after the embedding layer, and h_ℓ denote the hidden states on the *shortcut path* after the ℓ-th residual update. Each Transformer layer contains two residual sub-blocks (self-attention and MLP) wrapped by skip connections and normalisation. For analytical convenience, we view each residual sub-block as a *residual layer* $M_i(\cdot)$ that perturbs the shortcut vector:

$$h_0 = \text{Embedding}(\text{input}), \tag{1}$$

$$h_{i+1} = h_i + M_i(h_i), \quad i = 0, \dots, N-1, \tag{2}$$

$$\text{output} = \text{Linear}(\text{RMSNorm}(h_N)), \tag{3}$$

where N is the total number of residual layers (two per Transformer block in LLaMA-style models). In this view, the *shortcut stream* $\{h_i\}$ forms a global path that traverses the entire network, while each M_i applies a local transformation whose output is added back to the shortcut (Fig. 1).

This formulation highlights an important structural property: the model's internal computation can be seen as a *discrete-time dynamical system* on the residual stream. Starting from h_0, each step adds a new update $M_i(h_i)$ to the accumulated representation. Intuitively, for a well-behaved network, these updates should be "small" corrections that refine, rather than overwrite, information carried by h_i.

2.2. Directional stability and norm growth

To better understand how the residual stream evolves with depth, we analyse several 7–8B-parameter models (LLaMA-2-7B, LLaMA-3.1-8B and Baichuan-2-7B). For each residual sub-layer (attention and MLP), and for each token position, we measure:

- the *cosine similarity* between its input and output,

$$\cos(h_i, h_{i+1}) = \frac{h_i^\top h_{i+1}}{\|h_i\| \, \|h_{i+1}\|}, \tag{4}$$

- and the *L2 norm* $\|h_i\|$ along the shortcut path.

Averaged over a calibration dataset, we consistently observe the following:

1. **Directional stability.** Across almost all layers and models, the cosine similarity between h_i and h_{i+1} is very high, typically above 0.8 and often exceeding 0.98 in certain depth ranges (e.g., layers 23–29). This indicates that each residual layer changes the *direction* of the shortcut vector only mildly: the output h_{i+1} remains highly aligned with h_i.
2. **Monotonic norm growth.** In contrast, the norm $\|h_i\|$ shows a clear monotonic growth pattern as depth increases. For example, in Baichuan-2-7B, the normalized norm in MLP blocks roughly doubles between middle and deep layers, while similar trends are observed in the attention blocks and other models.

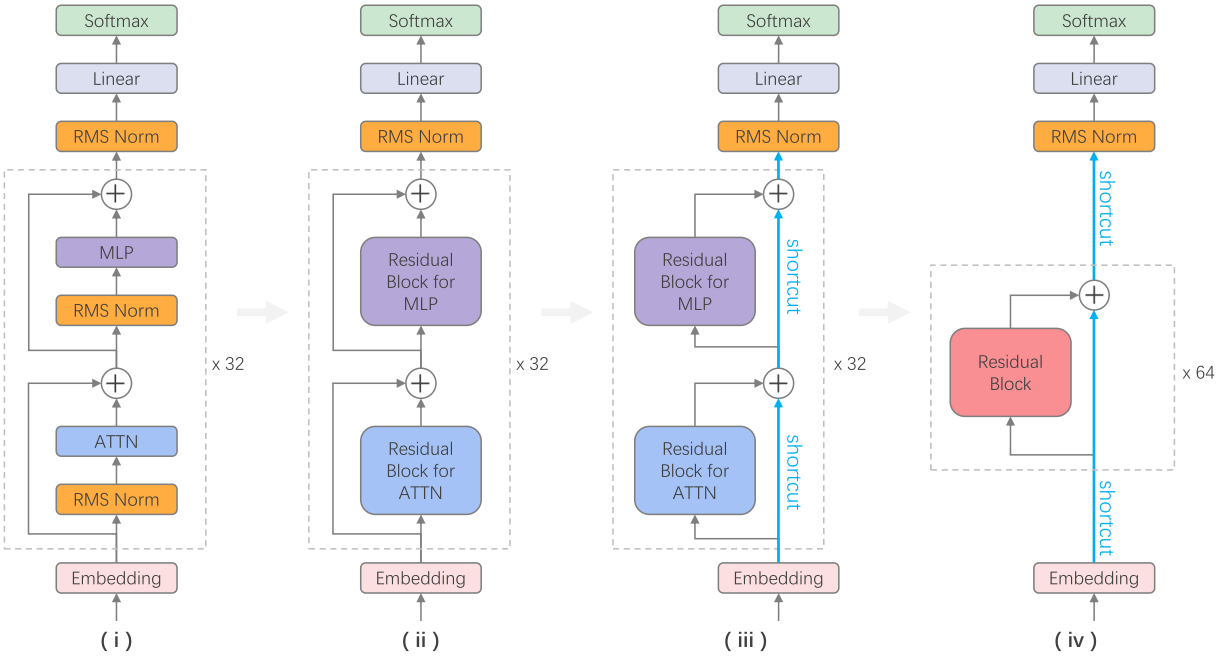


Fig. 1. (i) Structure of LLaMA 3.1. (ii) LLaMA structure with residual block representation. (iii) LLaMA structure with shortcut connection representation. (iv) A 32-layer LLM (like LLaMA 3.1) can be viewed as a 64-layer model consisting of residual blocks.

Taken together, these findings suggest that many residual layers behave as *amplitude modulators*: they significantly *rescale* the magnitude of the shortcut vector while only slightly bending its direction. In other words, the residual stream evolves such that:

- the *semantic direction* of the representation changes slowly and smoothly;
- the *amplitude* (norm) accumulates steadily as more residual updates are added.

From this perspective, deleting a residual layer does not merely remove a functional transformation; it also removes a step of *amplitude modulation* that would otherwise have contributed to the overall norm of the shortcut.

2.3. Expected vs. observed effect of pruning

Given the above, one might expect that deep structured pruning should have a relatively benign effect: if most residual layers are near-identity in direction and primarily modulate amplitude, then removing some of them should reduce the final norm and thus the model's *effective capacity*, but not fundamentally disrupt the information flow. Under this intuition, performance should degrade gradually as more layers are pruned.

However, empirical results in prior work and our own experiments show that this expectation is overly optimistic. When we prune contiguous blocks of residual layers beyond a moderate ratio, the model often *collapses*: its outputs become incoherent or degenerate, and task accuracy falls sharply. This behaviour indicates that the residual stream is more fragile than a simple "depth–capacity" picture would suggest.

In the next section, we argue that the missing piece in this picture is the *ratio* between the norms of the shortcut and residual branches. While the absolute norm $\|h_i\|$ grows smoothly with depth, pruning changes *which vector* is fed into each residual layer and hence changes the relative magnitudes of shortcut and residual contributions. This leads to what we call *amplitude mismatch*, which we identify as the central mechanism behind pruning-induced collapse.

3. Amplitude mismatch in structured pruning

We now formalise the notion of *amplitude mismatch* and explain how it leads to catastrophic degradation when residual layers are removed. We first introduce a simple view of residual updates as implicit mixing between shortcut and residual branches, and then analyse how pruning distorts these mixing weights.

3.1. Implicit mixing weights in residual updates

Recall that each residual update can be written as

$$h_{i+1} = h_i + M_i(h_i). \tag{5}$$

For a given token and layer, consider the two vectors being added: the *shortcut* h_i and the *residual update* $M_i(h_i)$. Their relative contribution to h_{i+1} is governed by their norms. To make this explicit, we define the following *implicit mixing weights*:

$$w_i^{\text{short}} = \frac{\|h_i\|}{\|h_i\| + \|M_i(h_i)\|}, \tag{6}$$

$$w_i^{\text{res}} = \frac{\|M_i(h_i)\|}{\|h_i\| + \|M_i(h_i)\|}. \tag{7}$$

These quantities describe how much of the new representation is inherited from the past (shortcut) versus contributed by the current residual block. In well-trained LLMs, our empirical observations indicate that:

- w_i^{short} remains non-negligible for almost all layers;
- $\|M_i(h_i)\|$ typically has a smaller norm than $\|h_i\|$, implying that each block acts as a *refinement*, not a full replacement.

Thus, in the dense model, the residual stream can be thought of as a sequence of steps where each block slightly adjusts the representation while preserving a stable proportion of shortcut information.

3.2. Definition of amplitude mismatch

Deep structured pruning modifies this balance in a non-trivial way. When a block is removed, the input to subsequent residual layers

changes: instead of receiving the shortcut vector produced at its original depth, they now operate on a shallower shortcut vector with a different norm.

To capture this effect, consider a model before and after pruning. In the dense model, at some depth k, we have:

$$h_{k+1}^{\text{dense}} = h_k + M_k(h_k), \tag{8}$$

with mixing weights w_k^{short} and w_k^{res} determined by $\|h_k\|$ and $\|M_k(h_k)\|$ as above.

Suppose now that we prune a contiguous block of layers including M_k. After pruning, the computation at the same graph position is carried out by a different residual block $\tilde{M}_k$ acting on a different shortcut vector k'_k, for example the shortcut that would have occurred earlier in the dense model. The new update is

$$h_{k+1}^{\text{pruned}} = h'_k + \tilde{M}_k(h'_k). \tag{9}$$

with corresponding mixing weights:

$$R_k^{\text{dense}} = \frac{\|h_k\|}{\|M_k(h_k)\|}, \qquad R_k^{\text{pruned}} = \frac{\|h'_k\|}{\|\tilde{M}_k(h'_k)\|}. \tag{10}$$

We say that amplitude mismatch occurs when the ratio between shortcut and residual norms at a given depth is strongly distorted by pruning.

Definition 3.1 (Amplitude mismatch). Let h_k, M_k denote the shortcut and residual branch at depth k in the dense model, and $h'_k, \widetilde{M}_k$ the corresponding quantities after pruning. We say that pruning introduces *amplitude mismatch* at depth k if the norm ratio

$$R_k^{\text{dense}} = \frac{\|h_k\|}{\|M_k(h_k)\|} \qquad \text{and} \qquad R_k^{\text{pruned}} = \frac{\|h'_k\|}{\|\widetilde{M}_k(h'_k)\|}$$

differ significantly, i.e.,

$$\left|\log R_k^{\text{pruned}} - \log R_k^{\text{dense}}\right| \gg 0.$$

Equivalently, the implicit mixing weights $(w_k^{\text{short}}, w_k^{\text{res}})$ and $(\widetilde{w}_k^{\text{short}}, \widetilde{w}_k^{\text{res}})$ become substantially different.

Intuitively, amplitude mismatch means that the *relative strength* of the shortcut and residual vectors has changed. If $\|h'_k\|$ becomes too small compared to $\|\widetilde{M}_k(h'_k)\|$, the shortcut is effectively muted; if it becomes too large, downstream residual updates are suppressed.

3.3. Amplitude mismatch as a mechanism for pruning collapse

We now connect amplitude mismatch to the catastrophic degradation observed in deep structured pruning. The key idea is that the residual stream in a dense LLM maintains a *delicate balance*: each block is trained to operate within a particular range of shortcut norms and to mix new information with the accumulated context at a specific ratio. Pruning changes both the input norm and the *identity of the residual block* that operates at each depth.

Consider the following simplified argument.

Proposition 1 (Informal). *Assume that in the dense model there exist constants $0 < \alpha \le \beta < \infty$ such that for all layers i,*

$$\alpha \le \frac{\|h_i\|}{\|M_i(h_i)\|} \le \beta,$$

so that the shortcut always retains a non-trivial mixing weight w_i^{short}. Suppose that pruning yields a configuration where, at some depth k,

$$\frac{\|h'_k\|}{\|\widetilde{M}_k(h'_k)\|} \ll \alpha,$$

i.e., the shortcut is much weaker than the residual update. Then the updated state

$$h_{k+1}^{pruned} = h'_k + \widetilde{M}_k(h'_k)$$

is dominated by $\widetilde{M}_k(h'_k)$, so that most information encoded in h'_k is effectively overwritten. If such mismatches occur at multiple depths, the accumulated effect is a drastic distortion of the residual stream, which manifests as an abrupt breakdown of model behaviour.

This proposition formalises the intuition that **LLMs rely on a consistent shortcut–residual balance**. In the dense model, each block behaves like a small correction with bounded influence: it nudges the representation towards the target while preserving a substantial component of the past. Deep structured pruning breaks this balance in two coupled ways:

1. It **changes the shortcut norm** at each depth by skipping amplitude-modulating blocks, so that downstream layers receive inputs with norms for which they were not trained.
2. It **reassigns residual blocks** to shallower shortcut states: a block originally trained to refine a high-norm representation is now applied to a lower-norm one, amplifying its relative effect.

When the resulting shortcut–residual norm ratio deviates too far from the regime seen during training, the residual updates cease to be gentle refinements and instead become **destructive overwrites**. This explains why, beyond a certain pruning ratio, model performance collapses rapidly rather than decaying smoothly.

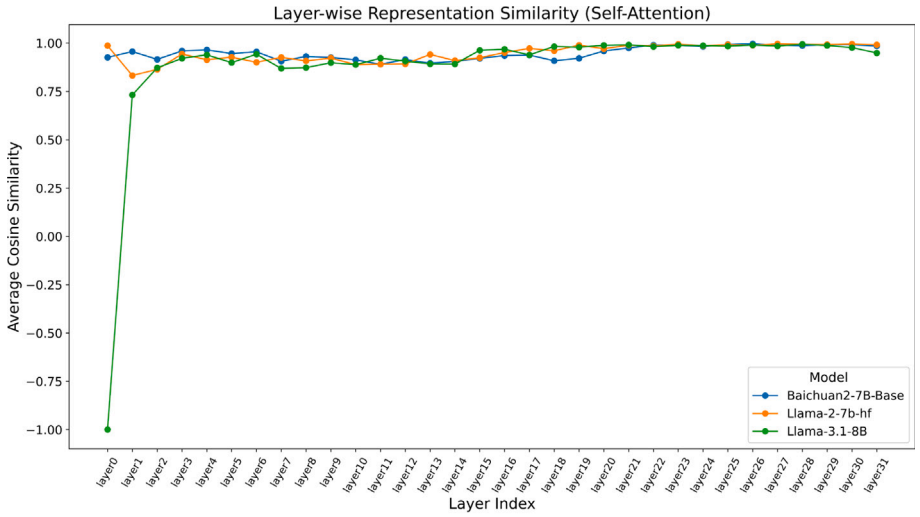
Our empirical results in later sections corroborate this view:

- Direct deep pruning without compensation leads to severe performance drops and unstable behaviour.
- Once we explicitly restore the original shortcut norm at each pruned position—by inserting a simple scalar Aggregator $h \mapsto \lambda_i h$ that matches the dense model's amplitude—the model no longer collapses. Removing more layers then results in a *predictable decrease in accuracy*, consistent with the loss of capacity, but the model remains functional and coherent.

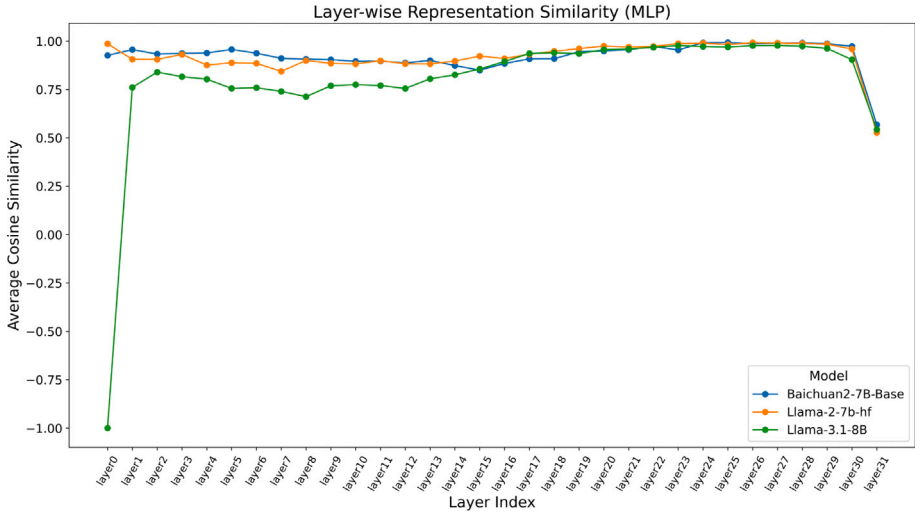
3.4. Numerical analysis of amplitude mismatch

For each residual sub-layer (attention or MLP), we measure the cosine similarity between its input h_i and output h_{i+1} on the calibration dataset. As shown in Fig. 2, the similarity stays consistently high (typically above 0.8, and in layers 23–29 often exceeding 0.98) across almost all layers and models, indicating that each residual block only mildly changes the *direction* of the shortcut vector: h_{i+1} remains highly aligned with h_i. In contrast, Fig. 3 reports the normalized L2 magnitude of the hidden states across depth and reveals a clear monotonic growth pattern. For example, in the MLP blocks of Baichuan2-7B the normalized norm increases from roughly 60–70 around layer 23 to over 140 by layer 29, while LLaMA-3.1-8B grows from about 20 to more than 40 over the same range. Similar trends appear in the self-attention blocks. Thus, deeper layers substantially amplify the *norm* of the representation while leaving its direction nearly unchanged. These observations suggest that many residual modules primarily act as amplitude modulators along the shortcut path—strongly rescaling the hidden state while making only very tiny directional adjustments, so that changes in most individual dimensions are extremely small and can be regarded as noise ignorable outputs.

This evidence supports our claim that **amplitude mismatch is the core mechanism behind pruning-induced collapse in LLMs**. In the next section (Section 4), we build on this insight to design AEMC, a training-free pruning framework that uses per-layer, per-task Aggregator scalars λ_i to correct amplitude mismatch while keeping the residual stream direction intact.



(a) Self-attention sub-layer.



(b) MLP sub-layer.

Fig. 2. Layer-wise cosine similarity between the input and output of each residual sub-layer (self-attention vs. MLP) in 32-layer LLMs.

4. Proposed method: Aggregator-efficient model compression (AEMC)

In this section we present **Aggregator-Efficient Model Compression (AEMC)**, a training-free framework for deep structured pruning of LLMs. AEMC is built directly upon the amplitude mismatch analysis in Section 3: rather than retraining the pruned model, we explicitly restore the residual norm dynamics by inserting simple Aggregator blocks at pruned locations.

4.1. Design goals and high-level idea

AEMC is designed to satisfy three goals:

1. **Correct amplitude mismatch.** The primary objective is to restore, after pruning, the *shortcut–residual norm ratio* that the dense model would have produced, thereby preventing catastrophic overwriting of accumulated information.
2. **Remain strictly training-free.** The method must not introduce any learnable parameters or require gradient-based optimisation. All operations should be computable from a small calibration set and the original dense model.
3. **Be simple and deployment-friendly.** The pruned model should retain the same basic architecture as the original model, with only minimal modifications that are easy to implement in existing inference pipelines.

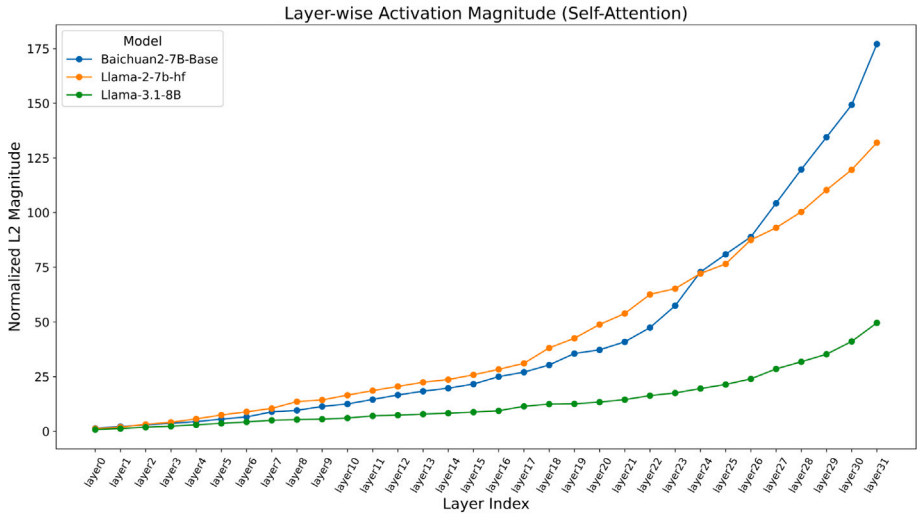
To this end, AEMC proceeds in two conceptual steps:

- **Layer selection.** We first determine which residual blocks are candidates for removal, using a similarity-based redundancy score consistent with prior deep pruning work.
- **Amplitude compensation.** For each removed block, we insert a *parameter-free scalar Aggregator* that rescales the shortcut vector by a layer- and task-specific factor λ_i, estimated from a handful of calibration examples from the target task. This restores the shortcut norm that the dense model would have produced at the same depth.

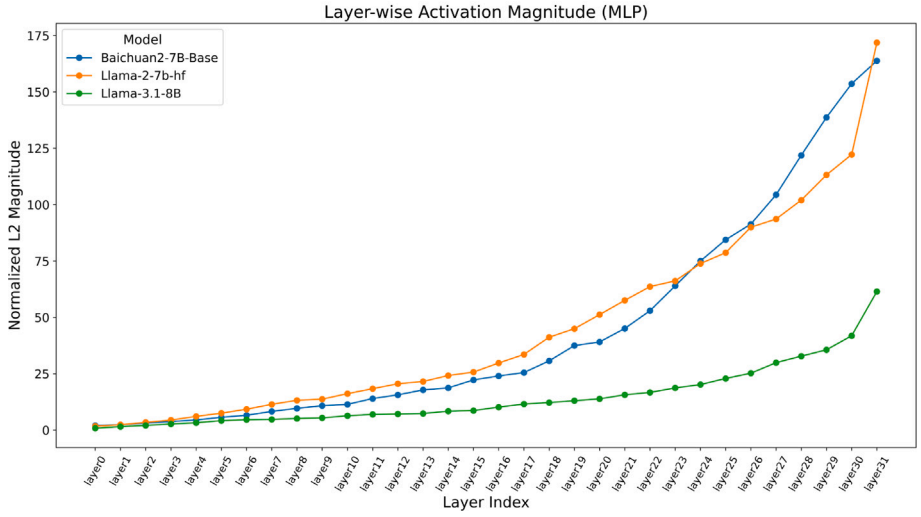
4.2. Aggregator blocks

Let h_i denote the shortcut vector at depth i in the dense model and M_i the corresponding residual block, so that

$$h_{i+1} = h_i + M_i(h_i).$$



(a) Self-attention sub-layer.



(b) MLP sub-layer.

Fig. 3. Layer-wise normalized magnitude (L2 norm) of hidden states for each residual sub-layer (self-attention vs. MLP) in 32-layer LLMs.

If we remove M_i during pruning, a naive approach would simply bypass it:

$$\widetilde{h}_{i+1} = h_i.$$

This eliminates both the functional effect of M_i and its contribution to the shortcut norm. As argued in Section 3, repeatedly doing so leads to *amplitude mismatch*, since downstream residual blocks now act on a shortcut whose norm is significantly lower than expected.

To correct this, AEMC replaces each removed block M_i with an **Aggregator block** AGG_i defined as

$$\widetilde{h}_{i+1} = \mathrm{AGG}_i(h_i) = \lambda_i h_i,$$

where $\lambda_i > 0$ is a scalar specific to layer i and the current task.

Several design choices are worth emphasising:

- **Scalar, not vector or matrix.** We deliberately choose λ_i to be a *single scalar* shared across all hidden dimensions and tokens at layer i. Since amplitude mismatch is ultimately a problem of norm distortion, it suffices—and is in fact desirable—to adjust only the magnitude of the shortcut vector while leaving its direction unchanged. More expressive parameterizations (e.g., per-dimension or matrix-valued scalings) would entangle amplitude correction with feature transformation and typically require learning, which conflicts with our training-free design.
- **Layer- and task-specific.** Different layers contribute differently to residual norm growth, and the same layer may exhibit slightly different norm gains across tasks. Therefore, each λ_i is estimated *per layer and per task* from a small calibration set, rather than being shared globally.

Under this construction, the Aggregator block can be seen as an **optimal simple repair**: it restores the shortcut norm that the dense model would have produced at depth $i+1$, while preserving the original representation direction and adding no new parameters.

4.3. Estimating layer-wise scalars λ_i

We now describe how to estimate the Aggregator scalars λ_i for a given model and task. The procedure is simple and entirely training-free.

Let f denote the dense LLM with residual layers $\{M_i\}_{i=0}^{N-1}$. We draw a small set of K calibration sequences $\{x^{(k)}\}_{k=1}^{K}$ from the target task distribution. For each sequence, we run the dense model once and record the shortcut vectors $\{h_i^{(k)}\}_{i=0}^{N}$.

For each layer i, we compute the **norm gain** between consecutive shortcut states:

$$g_i^{(k)} = \frac{\|h_{i+1}^{(k)}\|}{\|h_i^{(k)}\|}.$$

We then define

$$\lambda_i = \frac{1}{K}\sum_{k=1}^{K} g_i^{(k)},$$

i.e., the average multiplicative increase in the shortcut norm induced by the i-th residual block in the dense model.

Thus, if we remove block M_i, the Aggregator $\mathrm{AGG}_i(h_i) = \lambda_i h_i$ approximately restores the shortcut norm $\|h_{i+1}\|$ that the dense model would have produced.

In practice, we find that using **as few as 3–4 representative examples per task** suffices to obtain highly stable estimates of λ_i.

Beyond recovering norms, the calibration process also provides a **stability signal** for each layer: if the values $\{g_i^{(k)}\}_{k=1}^{K}$ vary widely across calibration examples, this suggests that the layer's behaviour is strongly input-dependent, and pruning it may be risky. Conversely, if $g_i^{(k)}$ is tightly concentrated, the layer behaves like a consistent amplitude modulator and is a good candidate for removal.

In our implementation, we exploit this observation as a practical heuristic: layers whose norm gains show large variance on the calibration set are marked as *unsafe to prune* and are retained even if they appear redundant under a purely similarity-based criterion.

4.4. Layer selection and integration with existing pruning schedules

AEMC is agnostic to the specific pruning schedule, i.e., the strategy used to select which residual blocks to remove. To isolate the effect of amplitude compensation and facilitate comparison with prior work, we adopt a **cosine-similarity–based** schedule consistent with previous deep pruning methods such as SliceGPT and LLM-Streamline.

Concretely, for each residual layer M_i, we measure its **redundancy score** on the calibration set as

$$r_i = 1 - \mathbb{E}_{x,t}\left[\cos(h_i^{(x,t)},\, h_{i+1}^{(x,t)})\right],$$

where the expectation is taken over calibration sequences x and token positions t, and $\cos(\cdot,\cdot)$ denotes cosine similarity. Layers whose input and output are highly aligned (large cosine similarity) are considered more redundant and thus obtain lower r_i scores.

Given a target pruning ratio ρ, we sort layers by ascending redundancy score r_i (most redundant first) and select a subset P of layers whose removal accounts for approximately ρ of the total parameters. Layers in P are replaced by Aggregator blocks with their corresponding λ_i; all other layers are kept intact.

Because AEMC operates **on top of** existing pruning schedules, it can be straightforwardly combined with other layer-selection strategies, including block influence metrics, PCA-based redundancy, or task-specific saliency measures. The core idea remains the same: wherever a residual block is removed, we insert an Aggregator that restores the expected shortcut norm.

4.5. Algorithm summary

Algorithm 1 summarises the full AEMC procedure.

By design, all steps in AEMC are **training-free**: we never update model weights or introduce new learnable parameters. The only data-dependent computation is the estimation of λ_i and redundancy scores from a small calibration set.

Algorithm 1 AEMC: Training-free deep pruning with Aggregator blocks.

Require:
- Pretrained LLM f with residual layers $\{M_i\}_{i=0}^{N-1}$;
- Target dataset D;
- Pruning ratio ρ;
- Calibration size K.

Ensure: Compressed model $\widetilde{f}$.

```
Sample K calibration sequences from D.
Run the dense model f on these sequences and record shortcut vectors {h_i^(k)} for all layers i and examples k.
for each layer i do
    Compute norm gains g_i^(k) = ||h_{i+1}^(k)|| / ||h_i^(k)||.
    Set λ_i = (1/K) Σ_{k=1}^{K} g_i^(k).
    Compute redundancy score r_i from cosine similarity between h_i^(k) and h_{i+1}^(k).
    Optionally mark layer i as unsafe to prune if {g_i^(k)} has high variance.
end for
Sort layers by redundancy score and select pruning set P achieving ratio ρ while excluding unsafe layers.
Construct f̃ by replacing each M_i with AGG_i(h) = λ_i h for all i ∈ P.
return f̃.
```

5. Experiments

We evaluate AEMC on multiple open-source LLM families and a diverse set of NLP benchmarks. Our experiments aim to answer the following questions:

1. How does AEMC compare to existing **training-free** and **post-training** pruning methods in terms of task performance?
2. Does correcting amplitude mismatch yield competitive performance at *moderate to deep pruning ratios*?
3. What are the **memory** and **inference speed** benefits of AEMC compared to dense and pruned baselines?

5.1. Models and tasks

We consider two widely used LLM families:

- **LLaMA family**: LLaMA-3.1-8B, LLaMA-2-7B and LLaMA-2-13B [16];
- **Baichuan family**: Baichuan-2-7B and Baichuan-2-13B [17].

Following prior work on LLM pruning, we evaluate a range of zero-shot and few-shot benchmarks that cover different capability categories:

- **Reasoning**: CMNLI [18], PIQA [19];
- **Language understanding and commonsense**: ChID [20], WSC [21];
- **Knowledge**: BoolQ [22];
- **Reading comprehension and multiple-choice understanding**: C3 [23], RACE-Middle and RACE-High [24].

All evaluations are conducted under the OpenCompass framework [25], using the same prompt templates as in previous pruning studies to ensure fair comparison with existing baselines.

5.2. Baselines and settings

We compare AEMC against several state-of-the-art pruning methods:

Training-free baselines:.

- SliceGPT [14];
- Untrained variants of LaCo [13] and LLM-Streamline [26], where pruning is applied but post-training is omitted.

Table 1
Performance of pruning methods on classification benchmarks for multiple LLMs.

Model	Method	Ratio	C3	CMNLI	CHID	BoolQ	WSC	PIQA	Race-M	Race-H	Average
LLaMA3.1-8B	Dense	0.00%	65.3	33.0	73.8	68.2	36.5	81.1	71.6	64.5	61.7
	LLMPruner^{PT}	–	–	–	–	–	–	–	–	–	–
	SliceGPT	23.9%	38.4	32.1	21.3	37.8	38.5	58.7	21.9	23.3	34.0
	LaCo	–	–	–	–	–	–	–	–	–	–
	LaCo^{PT}	–	–	–	–	–	–	–	–	–	–
	LLM-Streamline	24.4%	42.2	33.7	19.3	52.3	36.6	58.9	36.6	33.3	39.1
	LLM-Streamline^{PT}	24.4%	55.9	34.5	**54.5**	**67.6**	36.5	**74.5**	64.8	55.9	55.5
	AEMC	24.4%	**75.6**	**43.8**	24.5	67.0	**40.6**	67.6	**66.3**	**65.2**	**56.3**
Baichuan2-7B	Dense	0.00%	64.4	33.4	82.6	63.1	63.4	76.1	51.0	52.5	60.8
	LLMPruner^{PT}	–	–	–	–	–	–	–	–	–	–
	SliceGPT	22.2%	26.6	32.0	14.9	39.3	0.0	50.3	22.4	23.5	26.1
	LaCo	24.2%	50.9	33.0	76.2	56.2	26.9	68.5	27.7	29.0	46.0
	LaCo^{PT}	24.2%	58.0	32.9	**78.2**	55.9	3.85	**69.4**	26.3	25.0	43.7
	LLM-Streamline	24.2%	45.7	33.0	58.0	62.5	**36.5**	62.4	25.6	27.4	43.9
	LLM-Streamline^{PT}	24.2%	**60.4**	34.9	72.2	62.7	**36.5**	67.0	35.5	36.8	50.8
	AEMC	24.3%	57.9	**39.6**	77.1	**65.8**	**36.5**	50.2	**42.9**	**45.3**	**51.9**
LLaMA2-7B	Dense	0.00%	43.8	33.0	41.6	70.8	37.5	78.1	33.1	35.5	46.7
	LLMPruner^{PT}	24.8%	29.7	33.4	28.4	58.7	40.4	71.2	22.9	22.0	38.3
	SliceGPT	25.4%	31.5	31.6	18.5	59.9	**43.3**	68.3	27.0	29.4	38.7
	LaCo	27.0%	39.7	34.4	36.1	64.1	25.0	69.3	23.6	22.6	39.4
	LaCo^{PT}	27.0%	38.5	34.9	**38.1**	66.2	34.6	**73.2**	29.9	28.3	43.0
	LLM-Streamline	24.0%	40.2	34.4	21.5	67.3	40.3	69.0	**35.2**	34.7	42.8
	LLM-Streamline^{PT}	24.0%	43.3	33.0	24.1	67.5	36.5	71.5	34.8	**37.0**	43.5
	AEMC	24.4%	**44.2**	**38.1**	31.2	**69.8**	39.8	68.3	32.8	31.2	**44.4**
LLaMA2-13B	Dense	0.00%	47.5	33.0	47.2	71.5	51.0	79.8	60.0	58.1	56.0
	LLMPruner^{PT}	24.40%	29.6	33.0	29.5	58.0	47.1	72.7	21.9	22.5	39.3
	SliceGPT	23.60%	38.6	30.5	18.3	37.8	42.3	61.9	24.0	25.0	34.8
	LaCo	24.70%	45.0	32.9	**40.1**	64.0	35.6	**74.3**	56.5	54.5	50.4
	LaCo^{PT}	–	–	–	–	–	–	–	–	–	–
	LLM-Streamline	24.40%	47.0	33.0	36.5	62.3	**64.4**	73.5	60.2	58.3	54.4
	LLM-Streamline^{PT}	24.60%	45.7	33.0	38.0	66.2	36.5	74.0	58.6	57.4	51.2
	AEMC	24.60%	**51.2**	**38.1**	31.2	**67.4**	61.6	72.2	**62.4**	**59.0**	**55.4**
Baichuan2-13B	Dense	0.00%	65.6	33.2	86.7	66.8	42.3	78.1	68.9	67.2	63.6
	LLMPruner^{PT}	–	–	–	–	–	–	–	–	–	–
	SliceGPT	22.80%	25.0	30.4	10.4	37.8	0.0	51.0	21.5	21.5	24.7
	LaCo	24.70%	61.1	33.0	76.7	62.3	**60.6**	68.8	56.9	57.8	60.0
	LaCo^{PT}	–	–	–	–	–	–	–	–	–	–
	LLM-Streamline	24.70%	59.1	**34.4**	81.9	61.8	36.5	71.0	63.0	60.4	58.5
	LLM-Streamline^{PT}	24.60%	63.5	33.0	**84.1**	62.0	38.5	**72.0**	**70.2**	**66.3**	61.2
	AEMC	24.40%	**63.7**	33.2	79.2	**63.3**	49.3	69.7	67.5	65.6	**61.4**

^{PT} Denotes additional post-training/fine-tuning used after pruning.

Post-training baselines:.

- LLMPruner [11] with LoRA-based fine-tuning;
- LaCo [13] with calibration-guided merging and post-training;
- LLM-Streamline [26] with fine-tuned lightweight decoder blocks.

All methods operate at **similar pruning ratios**, typically removing $\sim 24\% - 25\%$ of the model parameters via deep structured pruning, consistent with prior settings. AEMC differs only in that:

- it replaces each removed block with an Aggregator block $h \mapsto \lambda_i h$ as described in Section 4;
- it uses a small calibration set ($K = 3$) per task to estimate λ_i and redundancy scores;
- it does not perform any gradient-based fine-tuning.

5.3. Experiments results

Table 1 presents the performance of AEMC across multiple datasets and model configurations, compared against both training-free and post-training pruning baselines.

AEMC achieves strong results across the board, outperforming training-free baselines and even surpassing post-trained methods. On every global metric (i.e., the average across datasets), AEMC achieves state-of-the-art (SOTA) performance. Moreover, AEMC achieves the best score on more than half of the individual dataset-level metrics, reinforcing its robustness across diverse tasks. Notably, all of these results are achieved without any fine-tuning, demonstrating the practical strength of the proposed amplitude compensation strategy.

Beyond average scores, AEMC demonstrates consistent performance across diverse datasets and model sizes on a broad set of controlled benchmarks (multiple-choice and classification-style evaluations). This consistency highlights the generality of the proposed amplitude compensation mechanism and its applicability across a wide range of real-world LLM deployments.

In some cases (e.g., on the CMNLI datasets), AEMC even slightly outperforms the unpruned Dense model. This counterintuitive phenomenon has also been reported in prior work [13,14], and is generally attributed to the pruning operation removing noisy or overly dominant components, thereby reducing overfitting or misalignment in specific prompt-task settings, a phenomenon often referred to as model delusion correction.

While AEMC shares a superficially similar strategy with LLM-Streamline (both remove multiple layers and insert lightweight replacements), there is a fundamental difference in how layers are selected. LLM-Streamline relies on extensive sampling over large datasets to identify less important layers. In contrast, AEMC is designed with real-world deployment in mind, using only minimal dataset-specific sampling to

Table 2
Peak GPU memory usage comparison for pruning methods on **Llama2-13B**.

Pruner	AEMC	LaCo	Dense	LLMPru.	SliceGPT
Memory (MB)	19 624	19 422	25 902	19 874	22 506

guide pruning decisions. As a result, the layers removed by AEMC do not necessarily align with those identified by LLM-Streamline, which may explain the divergence in performance trends despite structural similarity.

While several post-training baselines occasionally achieve the best score on individual benchmarks, our results show that AEMC is consistently the strongest method in the strictly training-free regime. Importantly, AEMC can be applied whenever the original pretrained model fits into GPU memory for inference: it does not require loading a larger teacher model or allocating additional memory for fine-tuning, which makes it practical even in constrained deployment environments. In contrast, post-training approaches such as LaCo-PT or LLM-Streamline-PT exploit extra labelled data and extended optimisation, often under more generous memory budgets, to obtain task-specific gains. At comparable pruning ratios, AEMC attains the highest average accuracy across benchmarks, indicating that it transfers robustly across diverse tasks rather than being tuned for a single dataset. This combination of resource-friendliness and strong average performance suggests that AEMC offers a favourable trade-off between practicality and accuracy for real-world LLM compression.

5.4. Memory occupation

Table 2 summarizes the peak GPU memory usage of AEMC, the dense model, and representative structured pruning baselines on **Llama2-13B** under the same hardware configuration. Compared with the dense model, AEMC achieves a substantial reduction in peak memory (25 902 MB → 19 624 MB, about 24%), which primarily comes from deep block removal that reduces the memory footprint of intermediate activations as well as the KV cache during decoding while keeping the remaining architecture unchanged.

Across pruning-based baselines, AEMC operates with a comparatively small memory budget and is close to the lowest memory entry in Table 2. Importantly, this memory efficiency is achieved together with stronger task performance preservation enabled by amplitude compensation, making AEMC particularly suitable for deployment settings where memory is a primary constraint and any additional fine-tuning is undesirable.

6. Ablation studies

In this section, we analyse the internal behaviour and design choices of AEMC, focusing on the role of the λ parameter and the sensitivity of model performance to shortcut amplitude modulation.

6.1. Stability of λ estimates

We begin by examining whether the layer-wise λ values (i.e., used in AGG layers) can be reliably estimated from a small number of examples. For this purpose, we compute λ values under the dense (unpruned) configuration for both LLaMA3.1-8B and Baichuan2-7B on the C3 dataset (Appendix B).

To validate the stability of λ, we perform a diagnostic analysis using 40 randomly sampled examples. The top panel of Fig. 4 shows the average λ values for each residual block (both attention and MLP layers). Notably, λ consistently remains above 1.0 (typically around 1.07), which indicates a steady increase in the amplitude of the shortcut vector as the network depth increases. This trend also supports the theoretically observed logarithmic growth behaviour of residual activations.

The bottom panel of Fig. 4 shows the standard deviation of λ across the 40 samples. The variance remains extremely low across all layers for both models, indicating that λ is highly stable within a given dataset. This concentration of variance suggests that even a small number of examples (such as the 3 used in AEMC) can yield reasonably representative estimates of layer-wise λ values. While this does not formally guarantee small-sample accuracy, it provides empirical justification for our lightweight estimation approach.

6.2. Effect of λ on individual layers

To further investigate the amplitude mismatch phenomenon, we conducted a controlled study by manually varying the λ values used in the AGG layers.

This experiment parallels prior work on direct pruning, which shows that removing a small number of layers typically results in limited performance degradation without retraining [13]. When $\lambda = 1$, the AGG layer passes the shortcut vector unchanged, replicating the baseline case without compensation. Decreasing λ downscales the shortcut, simulating the impact of information overwriting, whereas increasing λ creates an inverted mismatch, where the shortcut dominates the residual signal. At large values, this dominance effectively bypasses downstream residual transformations.

To test these effects, we individually removed each residual layer—excluding the first and last—and replaced it with an AGG layer using different λ values. The results are shown in Fig. 5, with additional plots provided in the Appendix.

The results show that reducing λ leads to substantial performance degradation, particularly when pruning early layers (e.g., MLP 1–16). This supports our hypothesis: weakened shortcut amplitude allows residual outputs to dominate, leading to loss of accumulated semantic information. These early-layer shortcuts are critical for maintaining representational continuity throughout the model.

Conversely, increasing λ beyond 2.0 also harms performance, as the amplified shortcut suppresses meaningful residual transformations. When $\lambda \leq 3$, the effect is equivalent to skipping all subsequent residual layers, resulting in a sharp accuracy drop.

It is interesting that deeper layers (e.g., 20–30) tend to be less sensitive to changes in λ (Appendix C). This is likely due to the fact that the layers preceding AGG copy their input shortcut vector to their output rather than output a modification signal, resulting in limited distortion but reinforcing functional redundancy.

However, this observed robustness should be interpreted with caution. Each ablation in our setup modifies only a single layer at a time. In practice, structured pruning often removes multiple consecutive residual blocks. In such cases, small λ estimation errors can accumulate, compounding the mismatch and resulting in severe performance degradation.

6.3. Calibration dataset size and stability of λ

In the **Methodology** section, we introduced how λ_i values are computed. In the previous parts of the ablation study, we visualised these values to explore their effect on performance. Here, we further validate the calibration process itself. Fig. 6 illustrates how λ_i values vary when computed from different numbers of samples on C3 [23] benchmarks. As shown, even a small number of calibration examples (1 or 3) yields λ_i estimates that closely match those computed from larger sample sets (25 or even 100). This confirms that AEMC only requires a handful of examples to produce stable pruning configurations, making it well-suited for real-world scenarios with limited task-specific data.

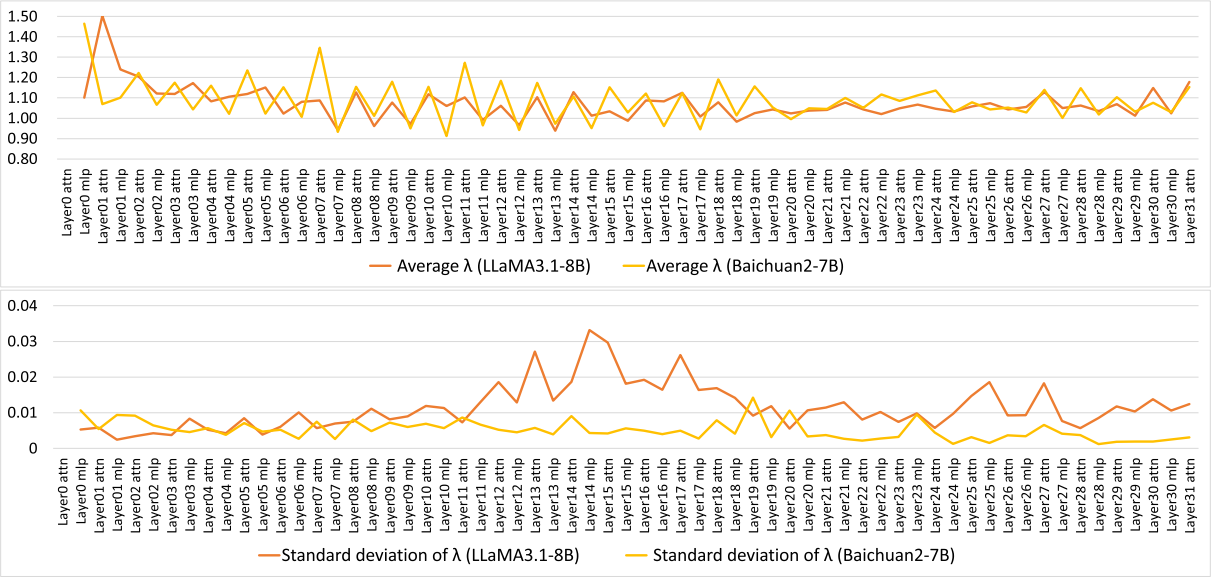


Fig. 4. (Top) λ values computed for each residual block in AEMC-LLaMA and AEMC-Baichuan on the C3 dataset, based on 40 randomly sampled examples. (Bottom) standard deviation of the λ values across these 40 examples.

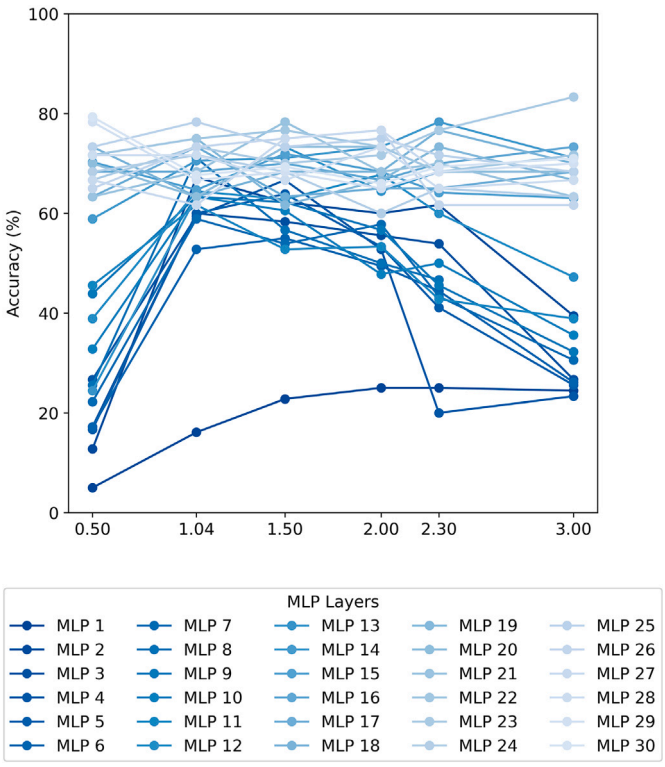


Fig. 5. Average performance of LLaMA3.1-8B on Race-H when replacing each individual MLP block with an AGG. Each line represents a specific residual block being replaced. The x-axis denotes different λ used in the AGG layer, and the y-axis represents the average task accuracy. See Appendix C for other results for multiple models and datasets.

7. Related work

We briefly review three lines of work related to AEMC: knowledge distillation, post-training quantisation, and structured pruning for LLMs. Our aim is to situate AEMC with respect to these approaches and highlight the remaining gap that amplitude-aware, training-free pruning fills.

7.1. Knowledge distillation

Compared with knowledge distillation, structural pruning methods such as AEMC do not require extra training, which makes them more suitable for low computing resource settings and avoids potential errors or biases introduced during training. Knowledge distillation compresses large teacher models into smaller students by transferring predictive behaviour, typically via softened logits or other intermediate signals [5–7]. Recent work has tailored distillation to LLMs, for instance by aligning token-level ranking [27] or jointly distilling final answers and chain-of-thought rationales [28].

While distillation can produce compact and performant models, it generally requires:

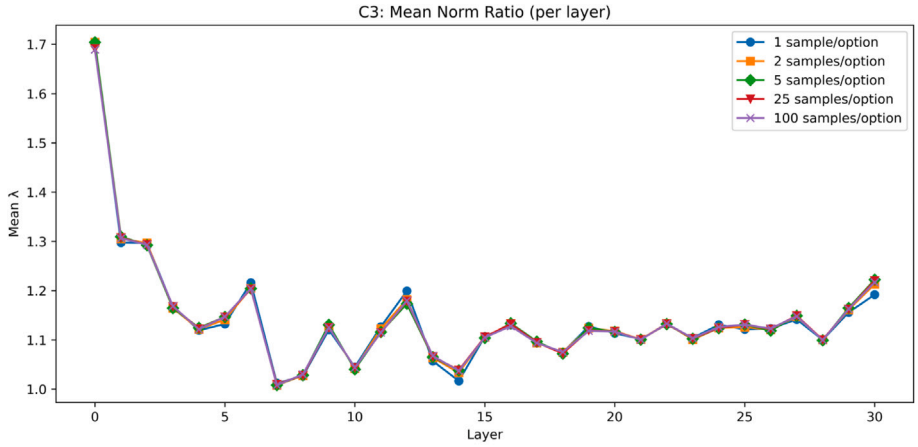
- substantial training data, often aligned with the target tasks;
- non-trivial training schedules and hyperparameter tuning;
- significant compute for both teacher and student optimisation.

These requirements limit the suitability of distillation for resource-constrained settings and for scenarios where a large teacher cannot be used during training, even though it may be available as a service.

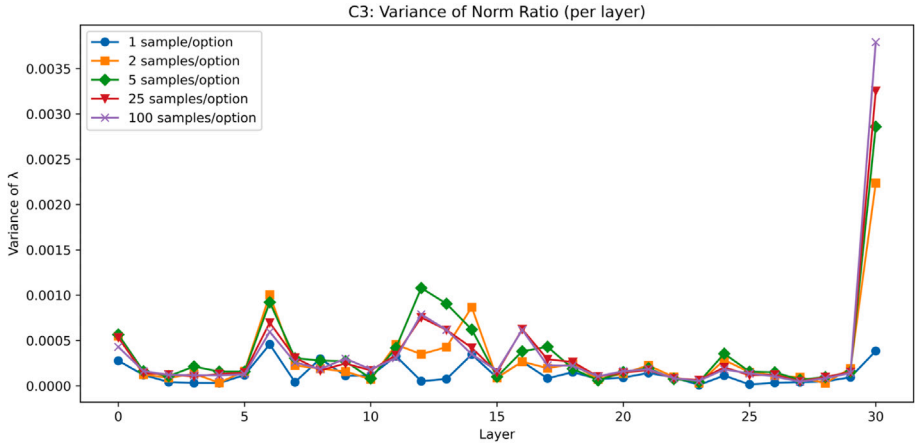
7.2. Post-training quantisation

Post-training quantisation (PTQ) reduces the numerical precision of model weights and activations in order to lower memory footprint and memory bandwidth requirements, while leaving the underlying network architecture unchanged. Modern calibration-based PTQ techniques, such as GPTQ, AWQ and QuaRot, attain high-fidelity 4-bit quantisation of LLaMA-style decoder models on standard language modelling and downstream benchmarks by using small calibration sets together with specialised outlier-aware design choices [29–31]. Benchmarks such as LLMC [32] provide systematic empirical comparisons of PTQ methods across model families, bit-widths and tasks.

However, PTQ primarily targets storage and memory bandwidth rather than computational depth. Quantised models retain the full number of layers of the original network, so the algorithmic FLOP count per



(a) Llama-3.1-8B Mean



(b) Llama-3.1-8B Variance

Fig. 6. Mean and variance of λ_i among different calibration dataset size. For other benchmarks, please refer to Appendix B.

generated token and the number of decoding steps remain unchanged, and any wall-clock latency improvements depend critically on the availability and quality of low-precision kernels on the target hardware. Furthermore, pushing PTQ to more aggressive settings and employing more sophisticated calibration pipelines typically incurs non-trivial implementation complexity and can require accelerators with substantial device memory to hold both the model and calibration batches, which may be misaligned with the resource constraints of small- to medium-scale deployments.

7.3. Structured pruning for LLMs

Structured pruning removes entire architectural units—such as attention heads, channels, blocks or layers—to achieve speedups that translate more directly into wall-clock gains on standard hardware. Early work on structured pruning for convolutional networks explored filter- and channel-level sparsity (e.g., StructADMM-style methods [33]); recent efforts have extended these ideas to LLMs.

LLMPruner [11] applies structured pruning to Transformer blocks and recovers performance using low-rank adaptation modules (LoRA) followed by fine-tuning. LaCo [13] merges each layer into its predecessor via a difference-preserving combination of weight matrices and uses calibration sets plus post-training to maintain output fidelity. LLM-Streamline [26] removes multiple layers and inserts lightweight decoder blocks that are then fine-tuned on task-specific data. SliceGPT [14] is one of the few training-free methods that can significantly reduce model width, its degradation is caused by both PCA-induced representation loss and the length mismatch introduced by width pruning, and in this work we focus specifically on analysing and correcting this length-mismatch effect.

While these methods demonstrate that deep pruning is possible, most rely on post-training to recover accuracy and do not explicitly analyse the residual norm dynamics that underlie pruning-induced collapse. In particular:

- Post-training approaches assume access to labelled data and sufficient compute for optimisation, which may be unavailable in SMA scenarios.
- Training-free approaches such as SliceGPT do not correct amplitude mismatch and can exhibit substantial degradation when pruning aggressively.

AEMC differs from prior work in two key respects:

- It is explicitly amplitude-aware: it views residual blocks as amplitude modulators and corrects the norm distortion introduced by pruning via simple Aggregator blocks.

- It is strictly training-free and parameter-free: Aggregators are determined entirely by scalar norm gains measured on a small calibration set, and no additional modules are learned.

To the best of our knowledge, AEMC is the first pruning framework that systematically treats amplitude mismatch as the core mechanism of pruning collapse in LLMs and addresses it without any retraining.

8. Discussion

In our experiments, we applied a fixed pruning ratio to enable fair comparison with existing benchmarks, i.e., removing 24%–25% of parameters across all tasks. While consistent, this uniform setting is not optimal in real-world deployment. Simpler tasks may tolerate more aggressive pruning, whereas more complex ones require retaining additional capacity. The performance variation observed across tasks supports this, suggesting that task-adaptive pruning could further enhance effectiveness.

Similarly, our experiments used default prompts under the OpenCompass framework to maintain consistency across baselines. While this allows fair benchmarking evaluation, it likely underutilises AEMC's full potential. Combining AEMC with task-specific prompt engineering could yield even stronger performance in practical use cases, without requiring model retraining.

AEMC is specifically designed to meet the needs of small-model application (SMA) scenarios, where developers often face resource constraints and cannot afford model fine-tuning. Real-world SMA systems typically integrate multiple lightweight LLM functionalities, such as user intent identification, comment filtering, and anomaly detection, within a single application. These functions differ in complexity, making a fixed pruning scheme suboptimal.

With AEMC, developers can assign task-specific λ values, store each model layer independently, and dynamically load only the required subset for each function. This modular prune–store–load pattern, though unconventional in model compression, maps naturally onto standard software engineering workflows. Instead of collecting large-scale datasets and fine-tuning models, SMA developers can repurpose existing test cases from software requirement documents to guide the selection of λ values. This shifts the adaptation process from data annotation and training to configuration and reuse, and the practices are familiar to most engineering teams.

Through extensive experiments, we demonstrate that the proposed AEMC method achieves SOTA performance within the category of training-free structural pruning, and even surpasses several baselines relying on post-training on some benchmarks. This again highlights the practical challenges that may be associated with post-training approaches, such as their sensitivity to hyperparameter tuning, dependence on the quality and alignment of the training data, and the additional engineering effort required to stabilize optimization. Moreover, post-training typically incurs substantially higher memory consumption than purely structural pruning, since it must store optimizer states, gradients, and intermediate activations, making such methods less suitable for deployment in scenarios with constrained computational or memory resources.

9. Conclusion

We present AEMC, a training-free pruning framework that addresses the critical issue of amplitude mismatch in residual connections. By introducing a lightweight Aggregator layer to preserve hidden state magnitudes, AEMC enables effective deep structured pruning without any additional training.

Both theoretical insights and empirical results demonstrate that AEMC maintains the residual information flow essential for model performance. Remarkably, AEMC not only outperforms all existing training-free pruning methods, but also achieves SOTA performance even when compared with pruning approaches that rely on post-training.

This makes AEMC a practical and highly competitive solution for real-world LLM deployment, particularly in scenarios where retraining is costly or infeasible.

CRediT authorship contribution statement

Mingrui Zhou: Writing – original draft, Validation, Methodology, Conceptualization. **Rui Mao:** Writing – review & editing, Writing – original draft, Supervision. **Somayajulu Sripada:** Writing – review & editing, Supervision, Methodology. **Xiao Li:** Writing – review & editing, Methodology, Project administration, Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Given Rui Mao role as Editor (Regular Section), he had no involvement in the peer review of this article and had no access to information regarding its peer review. Full responsibility for the editorial process for this article was delegated to another journal editor. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported by the FCDO/Innovate Ukraine–funded project UA Unity Hub (Project No. 10163874), awarded to Calling Bridge Limited, UK.

Appendix A. Explore layerwise hidden states

In the main paper, we discuss the phenomenon that the amplitude (i.e., vector norm) of the shortcut path in LLMs tends to increase progressively across layers. This observation aligns with previous studies such as [34,35], which independently reported similar trends in Transformer-based architectures.

To validate these findings in our experimental setting, we conducted a layerwise analysis of hidden state norms. The results are shown in Fig. A.7, where we specifically plot the shortcut vector at different residual blocks of LLaMA 3.1-8B.

As shown, the norm increases steadily from shallow to deeper layers. This trend supports our hypothesis in the main paper: residual outputs are progressively added to increasingly large shortcut vectors, so any mismatch in magnitude (such as those introduced by naive layer removal) can cause significant distortion in the final output.

This phenomenon is particularly relevant in methods like LLM-Streamline [26], which remove deep layers (e.g., layers 22–30 in LLaMA 3.1-8B) based on the BI score. Such pruning causes the input to layer 31 to come directly from the output of layer 21, bypassing multiple residual updates. Although the output distributions from layers 21 and 29 may exhibit similar **shapes** (i.e., peak locations), their **magnitudes** can differ substantially, as illustrated in Fig. A.7. This mismatch may lead to unstable token predictions or semantic drift, precisely the kind of "Amplitude Mismatch" we aim to address in this work.

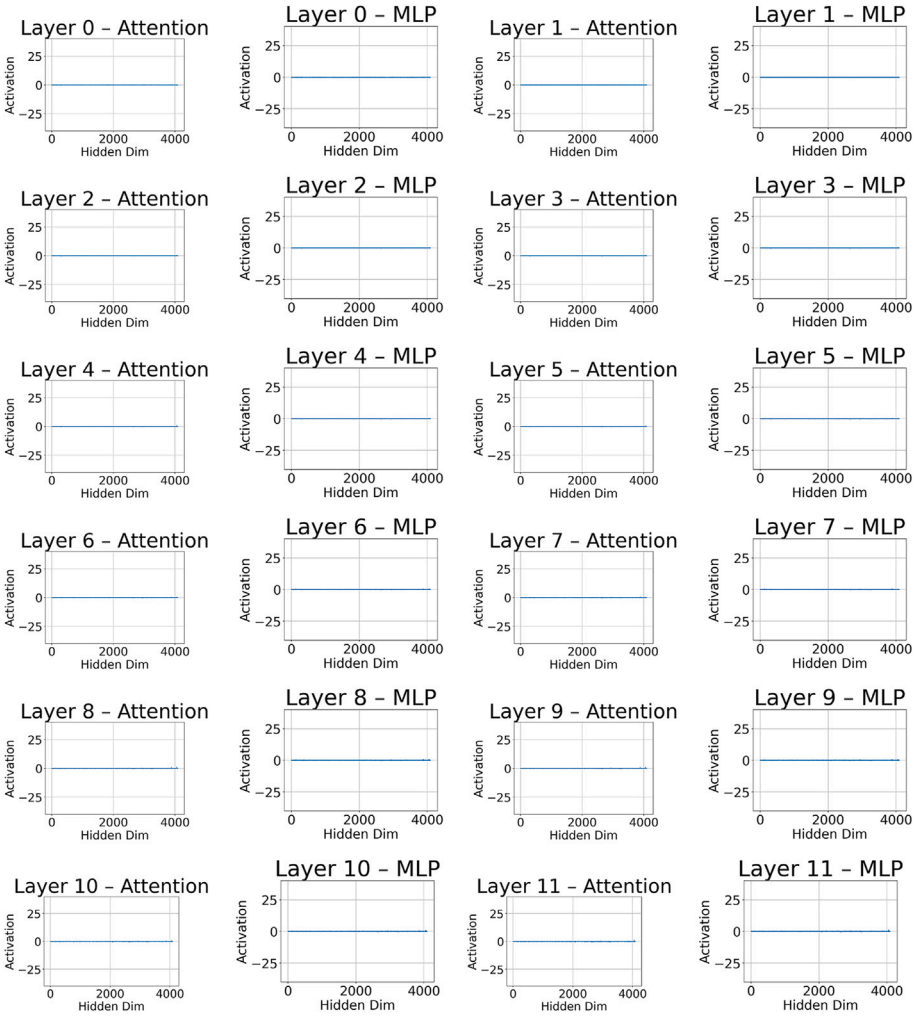


Fig. A.7. Magnitude enhancement at each layer's output (LLaMA 3.1-8B), Layers 0–11.

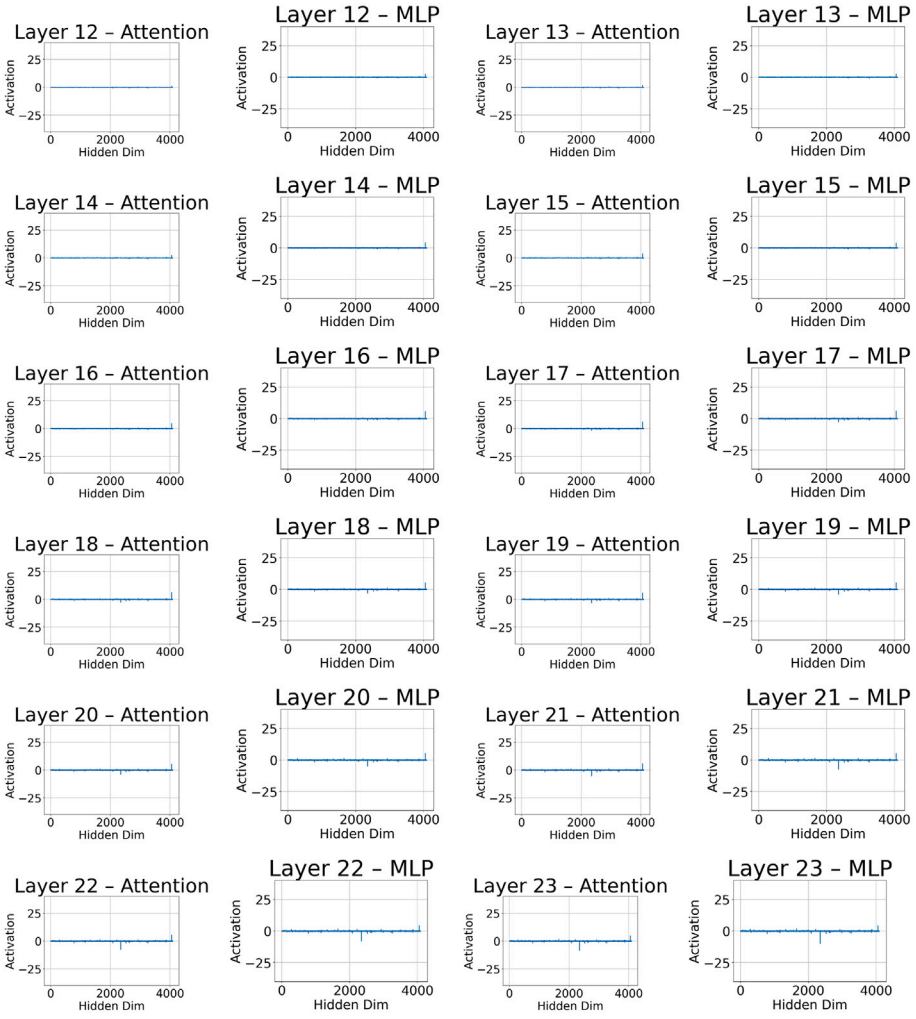


Fig. A.7. *(continued)* Layers 12–23.

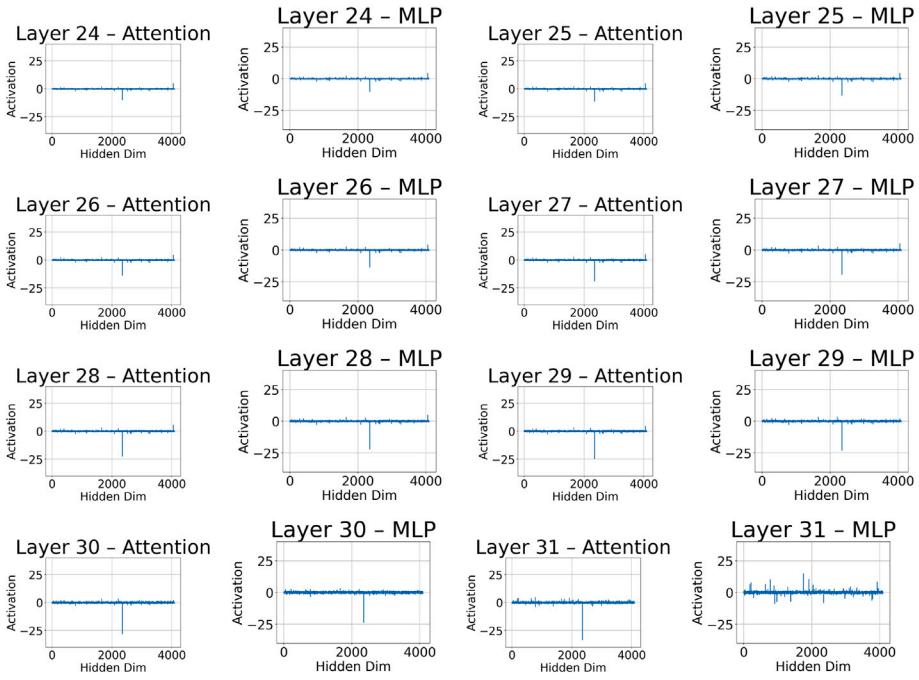


Fig. A.7. *(continued)* Layers 24–31.

Appendix B. Generalization to GPT style architectures

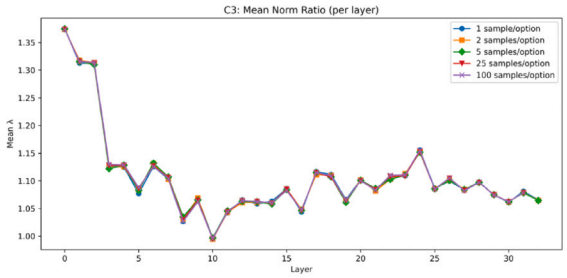
To further examine whether our conclusions generalize beyond the LLaMA and Baichuan families, we additionally evaluate AEMC on a reproduced GPT-style decoder-only Transformer, GPT-J-6B, using the same evaluation protocol and depth pruning setting as in the main experiments. As shown in Table B.3, AEMC maintains comparable performance on C3 and CMNLI, and shows relatively small degradation on WSC, while exhibiting a larger drop on CHID and RACE under the same depth reduction. Overall, these results provide supplementary evidence that AEMC is applicable to GPT-style architectures and that the observed behaviour under deep layer removal is not specific to a single model family.

Table B.3
Additional results on a reproduced GPT-style model (GPT-J-6B).

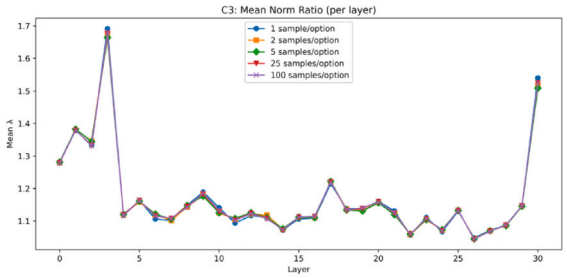
Benchmark	Dense GPT-J-6B (baseline)	AEMC (24.3%)
C3	43.78	40.22
cmnli	32.81	32.09
chid	37.61	27.62
WSC	67.31	65.38
piqa	75.79	70.13
race-middle	23.82	21.73
race-high	24.87	21.56

Appendix C. Calibration robustness and task-specific λ_i estimation

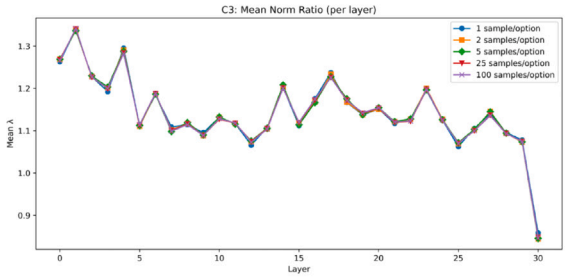
Fig. C.9 compares the λ_i values across multiple datasets used in this paper. The substantial variation between datasets confirms our assumption that task-specific calibration is necessary, i.e., different tasks benefit from different pruning strategies. This observation contrasts with the approach taken in prior work such as LLM-Streamline [26], which uses global sampling across multiple tasks to determine pruning decisions. Our results suggest that such unified sampling may overlook important task-level differences (Fig. C.8).



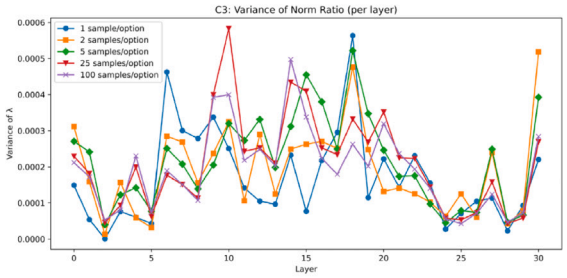
(a) Baichuan2-13B Mean



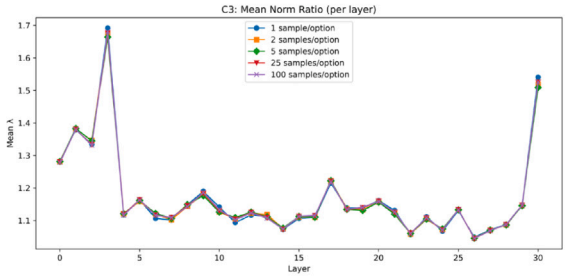
(b) Baichuan2-13B Variance



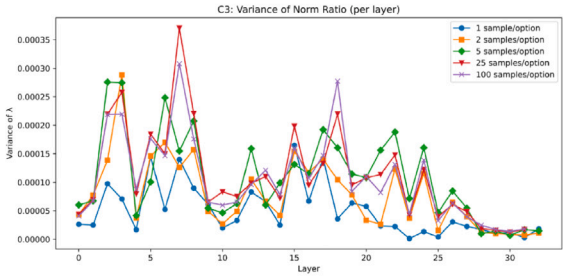
(c) Baichuan2-7B Mean



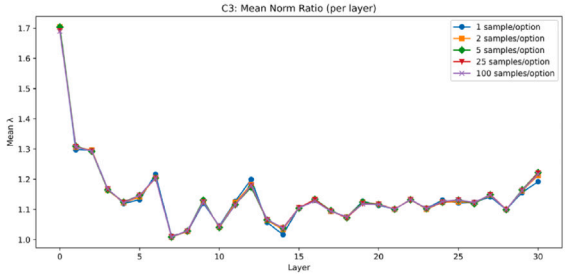
(d) Baichuan2-7B Variance



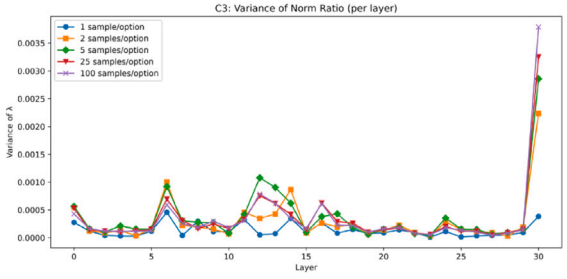
(e) Llama2-13 Mean



(f) Llama2-13 Variance

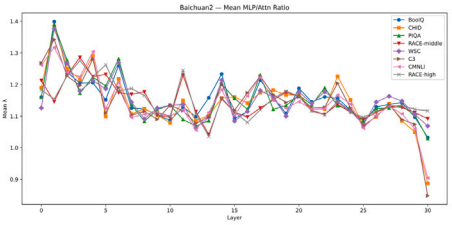


(g) Llama-3.1-8B Mean

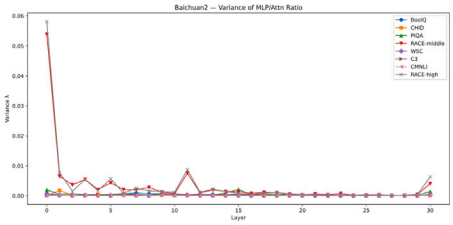


(h) Llama-3.1-8B Variance

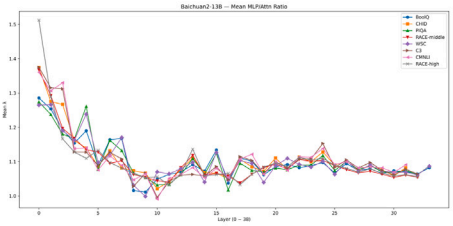
Fig. C.8. Comparison of mean and variance for various models on C3.



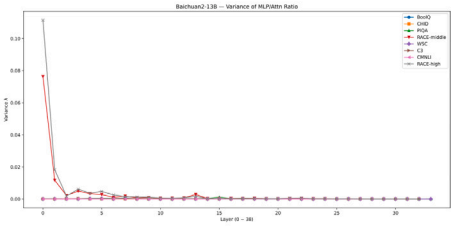
(a) Baichuan Mean (all benchmarks)



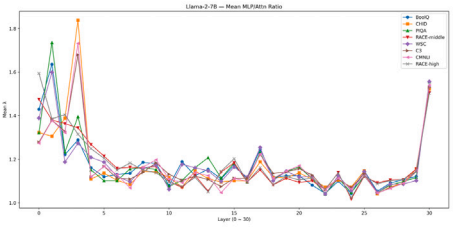
(b) Baichuan Variance (all benchmarks)



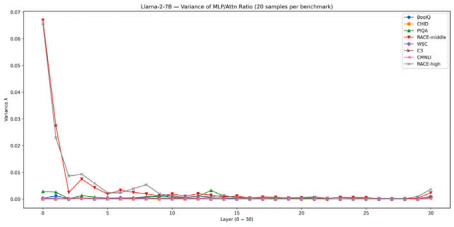
(c) Baichuan 13B Mean (all benchmarks)



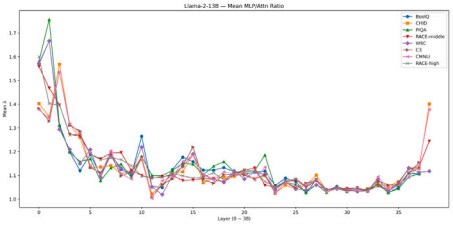
(d) Baichuan 13B Variance (all benchmarks)



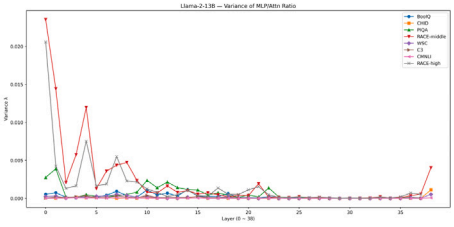
(e) Llama-2-7B Mean



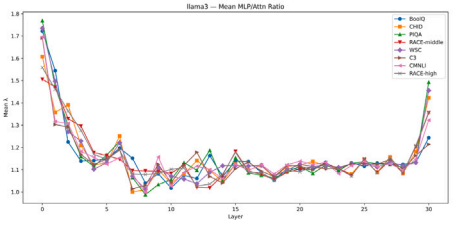
(f) Llama-2-7B Variance



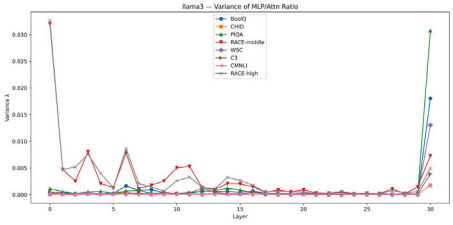
(g) Llama2-13B Mean (all benchmarks)



(h) Llama2-13B Variance (all benchmarks)



(i) Llama3 Mean (all benchmarks)



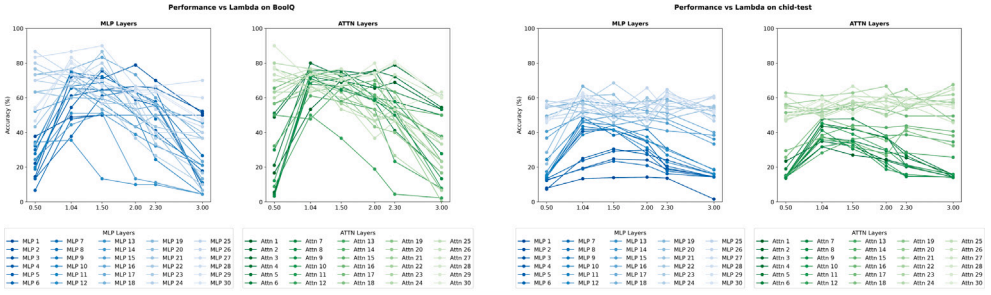
(j) Llama3 Variance (all benchmarks)

Fig. C.9. Mean and variance of λ_i for different models and benchmarks.

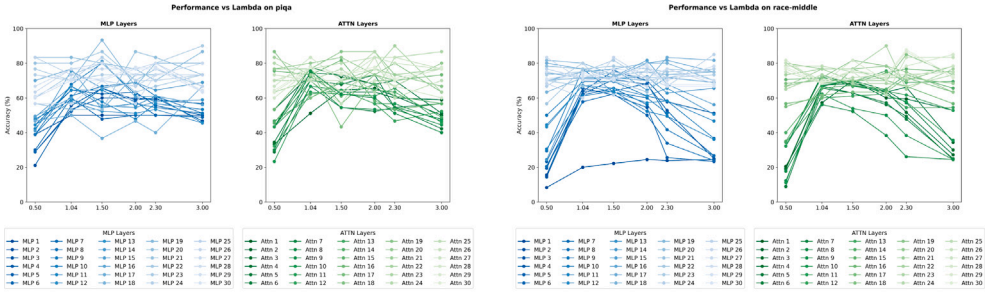
Appendix D. Evaluating layerwise impact of λ across tasks and models

Figs. D.10 and D.11 provide supporting details for Fig. 4 in the main paper. They illustrate the effect of replacing individual MLP or Attention layers with AGG blocks on two models: LLaMA 3.1-8B and Baichuan2-7B. This directly follows up on our analysis in Section **Effect of λ_i on Individual Layers**, where we discussed how amplitude mismatch can affect accuracy when layerwise residuals are modified.

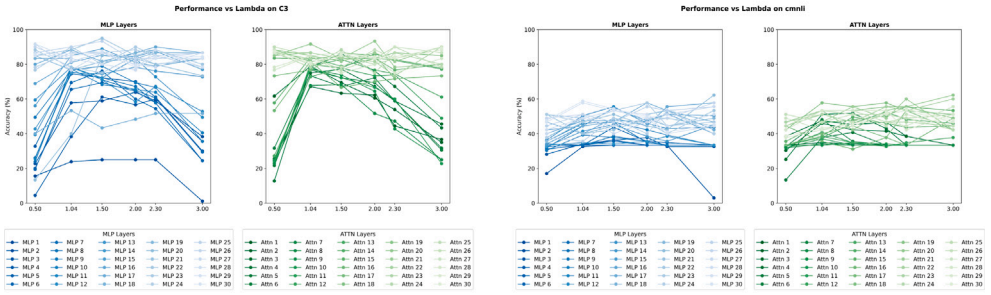
To conduct this experiment, we selected 150 examples for each dataset benchmark. These were used to evaluate the model's performance when replacing each residual block with an AGG layer at various λ values.



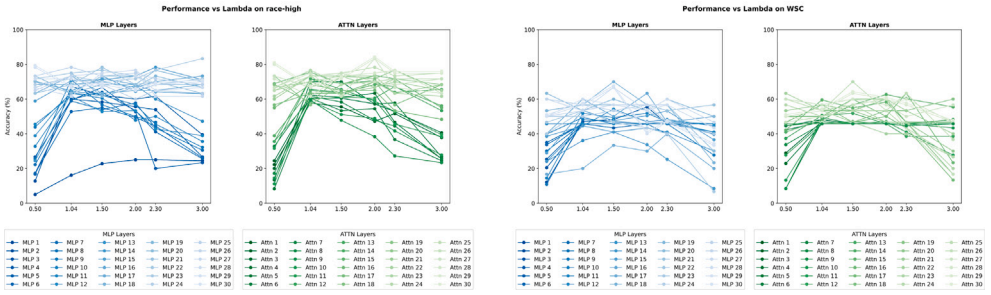
(a) BoolQ (b) ChID



(c) PIQA (d) RACE-Middle



(e) C3 (f) CMNLI

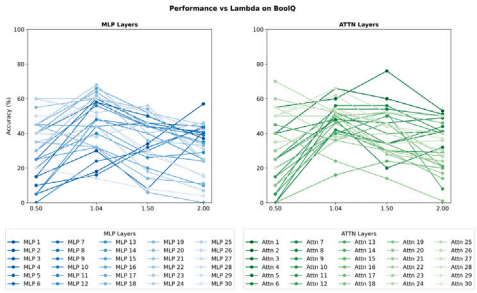


(g) RACE-High (h) WSC

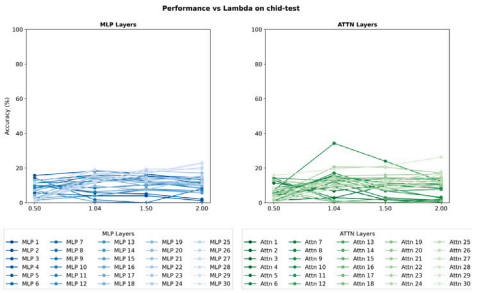
Fig. D.10. Layerwise modification results of LLaMA-3.1-8B across 8 benchmarks.

Due to the high computational cost of performing fine-grained layer replacement across two large models, the sampling in Figs. D.10 and D.11 remains relatively sparse. Nevertheless, the trends are clear: both excessively small and excessively large λ values cause substantial drops in accuracy. This confirms our earlier hypothesis that inappropriate λ settings, either under-amplifying or over-amplifying shortcut vectors, can lead to significant degradation in model performance.

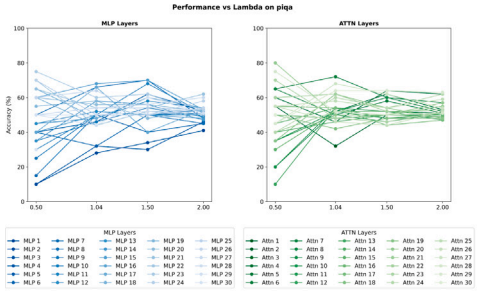
We also observe that optimal λ values generally lie between 1.0 and 1.5, which is consistent with the natural amplitude growth observed in residual streams. Furthermore, the results demonstrate that optimal λ values vary significantly across different tasks. This further justifies the need for task-specific calibration strategies, as using a single global λ would likely underperform in mixed-task or multi-functional deployments.



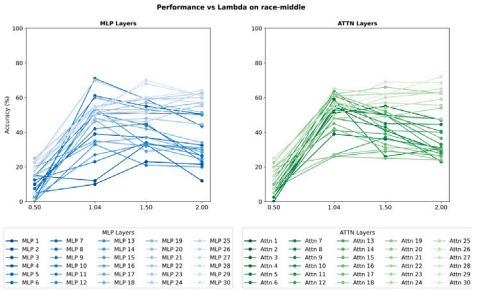
(a) BoolQ



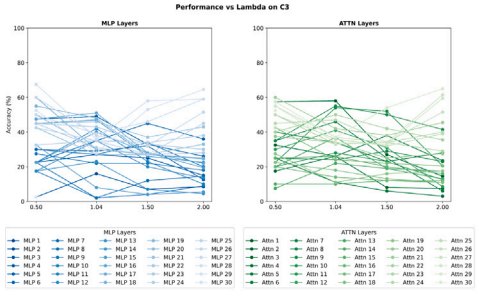
(b) ChID



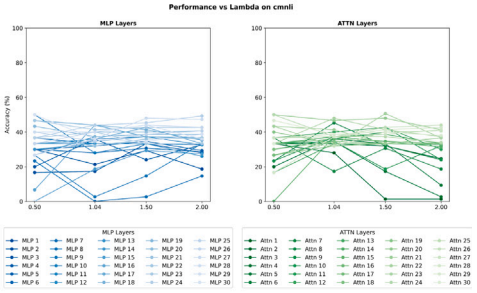
(c) PIQA



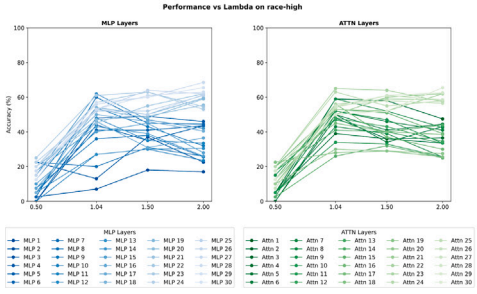
(d) RACE-Middle



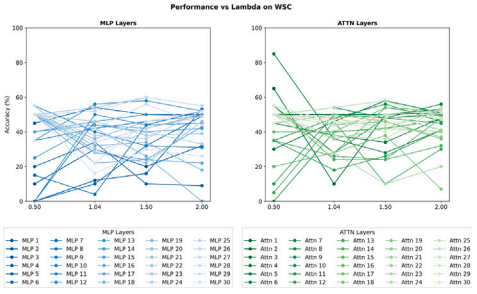
(e) C3



(f) CMNLI



(g) RACE-High



(h) WSC

Fig. D.11. Layerwise modification results of Baichuan2-7B across 8 benchmarks.

Data availability

Data will be made available on request.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).

[2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, Adv. Neural Inf. Process. Syst. 33 (2020) 1877–1901.

[3] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al., Deepseek-v3 technical report, arXiv preprint arXiv:2412.19437, 2024.

[4] R. Mao, G. Chen, X. Zhang, F. Guerin, E. Cambria, GPTEval: a survey on assessments of ChatGPT and GPT-4, in: Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), Torino, Italia, ELRA and ICCL, 2024, pp. 7844–7866.

[5] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, arXiv preprint arXiv:1503.02531, 2015.

[6] J. Gou, B. Yu, S.J. Maybank, D. Tao, Knowledge distillation: a survey, Int. J. Comput. Vis. 129 (6) (2021) 1789–1819.

[7] S. Li, J. Chen, Y. Shen, Z. Chen, X. Zhang, Z. Li, H. Wang, J. Qian, B. Peng, Y. Mao, et al., Explanations from large language models make small reasoners better, arXiv preprint arXiv:2210.06726, 2022.

[8] Z. Liu, Y. Wang, K. Han, W. Zhang, S. Ma, W. Gao, Post-training quantization for vision transformer, Adv. Neural Inf. Process. Syst. 34 (2021) 28092–28103.

[9] A. Gholami, S. Kim, Z. Dong, Z. Yao, M.W. Mahoney, K. Keutzer, A survey of quantization methods for efficient neural network inference, in: Low-Power Computer Vision, Chapman and Hall/CRC, 2022, pp. 291–326.

[10] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, Qlora: efficient finetuning of quantized llms, Adv. Neural Inf. Process. Syst. 36 (2023) 10088–10115.

[11] X. Ma, G. Fang, X. Wang, Llm-pruner: on the structural pruning of large language models, Adv. Neural Inf. Process. Syst. 36 (2023) 21702–21720.

[12] J. Ni, R. Mao, Z. Yang, H. Lei, E. Cambria, Finding the pillars of strength for multi-head attention, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), vol. 1, Toronto, Canada, Association for Computational Linguistics, 2023, pp. 14526–14540.

[13] Y. Yang, Z. Cao, H. Zhao, Laco: large language model pruning via layer collapse, in: Findings of the Association for Computational Linguistics: EMNLP 2024, 2024, pp. 6401–6417.

[14] S. Ashkboos, M.L. Croci, M.G. Nascimento do, T. Hoefler, J. Hensman, Slicegpt: Compress large language models by deleting rows and columns, arXiv preprint arXiv:2401.15024 2024.

[15] B.-K. Kim, G. Kim, T.-H. Kim, T. Castells, S. Choi, J. Shin, H.-K. Song, Shortened llama: Depth pruning for large language models with comparison of retraining methods, arXiv preprint arXiv:2402.02834 2024.

[16] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, et al., The llama 3 herd of models, arXiv preprint arXiv:2407.21783 2024.

[17] A. Yang, B. Xiao, B. Wang, B. Zhang, C. Bian, C. Yin, C. Lv, D. Pan, D. Wang, D. Yan, et al., Baichuan 2: Open large-scale language models, arXiv preprint arXiv:2309.10305 2023.

[18] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, T. Baldwin, Cmmlu: Measuring massive multitask language understanding in chinese, arXiv preprint arXiv:2306.09212 2023.

[19] Y. Bisk, R. Zellers, J. Gao, Y. Choi, et al., Piqa: reasoning about physical commonsense in natural language, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, 2020, pp. 7432–7439.

[20] C. Zheng, M. Huang, A. Sun, Chid: a large-scale chinese idiom dataset for cloze test, arXiv preprint arXiv:1906.01265 2019.

[21] H.J. Levesque, E. Davis, L. Morgenstern, The winograd schema challenge, KR 2012 (2012) 13th.

[22] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, K. Toutanova, Boolq: Exploring the surprising difficulty of natural yes/no questions, arXiv preprint arXiv:1905.10044 2019.

[23] K. Sun, D. Yu, D. Yu, C. Cardie, Investigating prior knowledge for challenging Chinese machine reading comprehension, Trans. Assoc. Comput. Linguist. 8 (2020) 141–155.

[24] G. Lai, Q. Xie, H. Liu, Y. Yang, E. Hovy, Race: Large-scale reading comprehension dataset from examinations, arXiv preprint arXiv:1704.04683 2017.

[25] OpenCompass Contributors, Opencompass: a universal evaluation platform for foundation models, 2023, https://github.com/open-compass/opencompass.

[26] X. Chen, Y. Hu, J. Zhang, Y. Wang, C. Li, H. Chen, Streamlining redundant layers to compress large language models, arXiv preprint arXiv:2403.19135 2024.

[27] T. Peng, J. Zhang, Enhancing knowledge distillation of large language models through efficient multi-modal distribution alignment, in: Proceedings of the 31st International Conference on Computational Linguistics, 2025, pp. 2478–2496.

[28] C.-Y. Hsieh, C.-L. Li, C.-K. Yeh, H. Nakhost, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, T. Pfister, Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes, in: Findings of the Association for Computational Linguistics: ACL 2023, 2023, pp. 8003–8017.

[29] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh, Gptq: Accurate post-training quantization for generative pre-trained transformers, arXiv preprint arXiv:2210.17323 2022.

[30] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han, Awq: activation-aware weight quantization for on-device LLM compression and acceleration, Proc. Mach. Learn. Syst. 6 (2024) 87–100.

[31] S. Ashkboos, A. Mohtashami, M.L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, J. Hensman, Quarot: outlier-free 4-bit inference in rotated llms, Adv. Neural Inf. Process. Syst. 37 (2024) 100213–100240.

[32] R. Gong, Y. Yong, S. Gu, Y. Huang, C. Lv, Y. Zhang, D. Tao, X. Liu, Llmc: benchmarking large language model quantization with a versatile compression toolkit, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, 2024, pp. 132–152.

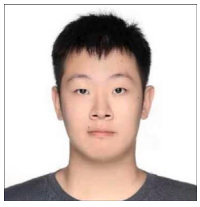
[33] T. Zhang, S. Ye, X. Feng, X. Ma, K. Zhang, Z. Li, J. Tang, S. Liu, X. Lin, Y. Liu, et al., Structadmm: achieving ultrahigh efficiency in structured pruning for dnns, IEEE Trans. Neural Netw. Learn. Syst. 33 (5) (2021) 2259–2273.

[34] S. Heimersheim, A. Turner, Residual stream norms grow exponentially over the forward pass, in: AI Alignment Forum, 2023, p. 23.

[35] H. Wang, S. Ma, L. Dong, S. Huang, D. Zhang, F. Wei, Deepnet: scaling transformers to 1,000 layers, IEEE Trans. Pattern Anal. Mach. Intell. 46 (10) (2024) 6761–6774.

[36] M. Zhou, X. Li, S. Sripada, R. Mao, ODSA: online differentiable structure adaptation for tiny TCN on IoT time series, in: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2026.

Author biography



Mingrui Zhou is a Ph.D. student at the University of Aberdeen. He received his B.S. degree from the University of Waterloo in 2020. His research focuses on efficient large language models, with an emphasis on model compression and fine-tuning methods.



Dr Rui Mao is a Research Scientist and Lead Investigator at Nanyang Technological University. He obtained his Ph.D. degree in Computing Science from the University of Aberdeen. His research interest lies at NLP, affective computing, and their applications in finance and cognitive science. He and his funded company (Ruimao Tech) have developed an end-to-end system (MetaPro) for computational metaphor processing and a neural search engine (wensousou.com) for searching Chinese ancient poems with modern language.



Dr Somayajulu Sripada is an Associate Professor of Computing Science at the University of Aberdeen. His research interests include natural language generation (NLG), particularly data-to-text NLG and its integration into data science and ML pipelines.



Dr Xiao Li received his Ph.D. in Computer Science from the University of Aberdeen, UK, in 2019. He is currently a Lecturer in Computer Science at the University of Aberdeen. His research interests lie primarily NLP and NLG, with a particular focus on statistical and neural approaches to text modelling, metaphor understanding and generation, and human–AI cognitive alignment. He has published extensively in leading international venues. In addition to his research activities, he is actively involved in teaching and supervision in areas related to artificial intelligence, robotics, and language technologies.